

Görsel Programlama Vize Tekrarı ve Pekiştirme

Görsel Programlama - Hafta 7



Görsel Programlama Vize Tekrarı ve Pekiştirme

Bu sunum, ilk altı haftada öğrenilen temel konuları pekiştirmeyi amaçlamaktadır.

IDE kullanımı, değişkenler, veri tipleri, kontroller, olaylar ve hata yönetimi gibi temel yapı taşlarına odaklanılacaktır.

Hedefimiz, bütünleşik bir proje üzerinde bu kavramları uygulamaktır ve vize sınavına hazırlık pratiği yapmaktır.



Pedagojik Hedefler: Neden Tekrar Yapıyoruz?

Öğrenciler, farklı konuları (IDE, kontroller, olaylar, değişkenler, dönüşümler, hatalar) tek bir projede birleştirebilmelidir.

Teorik bilgiyi pratik problem çözme senaryolarına dökme becerisi kazanılacaktır.

Vize öncesi yaygın hataları tespit etme ve giderme yeteneği geliştirilecektir.



Görsel Programlama Nedir? (Yeniden Tanım)

Görsel programlama, kullanıcı arayüzlerinin (GUI) sürükle-bırak yöntemleriyle tasarlanmasını sağlayan bir yaklaşımdır.

Temel olarak, kullanıcıların gerçekleştirdiği olaylara (event) yanıt veren kod bloklarının yazılması esasına dayanır.

Modern yazılım geliştirmede hız, prototipleme ve kullanıcı deneyimi açısından kritik öneme sahiptir.



Bölüm I: Visual Studio Arayüzüne Hakimiyet

Visual Studio, C# ve Görsel Programlama için standart IDE'dir. Temel paneller (Çözüm Gezgini, Özellikler Penceresi, Araç Kutusu) projenin yönetimini, kontrol özelliklerini ve bileşen eklemeyi sağlar. Bu panelleri etkin kullanmak, geliştirme hızını artırır.

Çözüm Gezgini (Solution Explorer)

Çözüm Gezgini, projedeki tüm dosyaları, referansları ve klasörleri hiyerarşik olarak gösterir.

Temel bileşenler: Form dosyaları (.cs), proje dosyaları (.csproj) ve ana çözüm dosyası (.sln).

Proje bağımlılıklarını ve sınıf yapılarını anlamak için merkezi bir noktadır.

Özellikler Penceresi (Properties Window)

Arayüzdeki her bir bileşenin (Label, Button vb.) özelleştirildiği yerdir. 'Name' (isim), 'Text' (görünen metin), 'BackColor' (arka plan rengi) ve 'Font' (yazı tipi) gibi temel özellikler buradan yönetilir.



Temel Kontrol: Label (Etiket)

Label, kullanıcıya bilgi vermek, sonuçları göstermek veya diğer kontrolleri açıklamak için kullanılır.
Genellikle kullanıcı tarafından değiştirilemez (statiktir).
Önemli özellikleri: 'Text' (içeriği), 'Name' (kod içinde erişim adı) ve 'ForeColor' (yazı rengi).

Temel Kontrol: TextBox (Metin Kutusu)

TextBox, kullanıcının veri girmesini sağlayan temel bileşendir.
TextBox.Text özelliği, kullanıcı tarafından girilen metni tutar.
Bu özelliğin her zaman 'string' (metin) tipinde olduğunu unutmamak kritiktir.



Temel Kontrol: Button (Buton)

Button, kullanıcı etkileşimi sonucu bir eylemi (olayı) tetiklemek için kullanılır. En yaygın olayı 'Click' olayıdır; bu olay gerçekleştiğinde belirlenen kod bloğu çalışır.

'Text' özelliği buton üzerinde yazan metni belirler.

Kontrol Özelliklerinin Yönetimi

Name özelliği, kontrolün kod içinde benzersiz bir şekilde tanımlanmasını sağlar (Örn: txtFiyat, btnHesapla).

Text özelliği, kontrolün kullanıcıya görünen metnini belirler.

BackColor ve Font gibi özellikler ise görsel estetiği ve okunabilirliği etkiler.

Bölüm II: Olay Güdümlü Programlama (Event-Driven Programming)

Görsel programlamanın temel mantığı olay güdümlü çalışmaktır. Kod, akışı sırayla takip etmek yerine, belirli olaylar (fare tıklaması, tuşa basılması, pencere açılması) gerçekleştiğinde tetiklenir.



Click Olayı ve İşleyiciler (Event Handlers)

'Click' olayı, bir butona tıklandığında gerçekleşen en temel olaydır. Bu olaya yanıt veren kod parçasına 'Olay İşleyici' (Event Handler) denir. Visual Studio, butona çift tıklandığında otomatik olarak 'btnName_Click' formatında bir olay işleyici metodu oluşturur.



Olay Mantığı Örneği: Metin Kutusu Nasıl Boşaltılır?

Bir butona tıklandığında, bir metin kutusunu boşaltmak basit bir atama işlemi gerektirir.

İlgili TextBox'ın 'Text' özelliğine boş bir string ataması yapılır.

Örn: `txtGiris.Text = ""` Bu, olay güdümlü programlamanın direkt etki prensibini gösterir.

Olay Mantığı Örneği: İki Metni Birleştirme (String Concatenation)

Kullanıcının girdiği iki metni birleştirmek için '+' operatörü kullanılır.
Örn: `string tamAd = txtAd.Text + " " + txtSoyad.Text;`
Sonuç bir Label kontrolüne atanabilir: `lblSonuc.Text = tamAd;`



Kullanım Senaryosu: Form Sıfırlama Butonu

Büyük formlarda, 'Sıfırla' butonu birden fazla TextBox, ComboBox veya diğer kontrolleri başlangıç durumuna döndürmelidir.
Bu, tek bir Click olayında birden çok kontrolün 'Text' özelliğinin olarak ayarlanmasını gerektirir.



Bölüm III: Değişkenler ve Temel Veri Tipleri

Değişkenler, program çalıştıkça değişebilen verileri geçici olarak bellekte saklamak için kullanılan isimlendirilmiş depolama alanlarıdır. Her değişkenin bir tipi (örneğin string, int, double) olmalıdır; bu tip, hangi tür veriyi tutabileceğini ve ne kadar yer kaplayacağını belirler.



Veri Tipi: String (Metin)

String, metinsel verileri (harfler, kelimeler, cümleler) depolamak için kullanılır. Metin kutularından okunan veriler (TextBox.Text) her zaman string tipindedir. C# dilinde stringler, çift tırnak (...) arasına yazılır.

Veri Tipi: Int (Tam Sayı)

Int, pozitif veya negatif tam sayıları (örneğin -10, 0, 150) depolamak için kullanılır.

Genellikle adet, sıra numarası, sayım gibi kesin sayısal değerler için idealdir. Int, dört bayt (32-bit) yer kaplar ve belirli bir aralıkta değer tutabilir.

Veri Tipi: Double (Ondalıkli Sayı)

Double, ondalıklı sayıları (gerçek sayıları) depolamak için kullanılır (örneğin 3.14, 18.5, 99.99).

Fiyat, ölçüm, yarıçap veya bilimsel hesaplamalar gibi hassasiyet gerektiren alanlarda tercih edilir.

Int'ten daha fazla yer kaplar ve daha geniş bir aralık ve daha yüksek bir hassasiyet sunar.



Veri Tipleri Arasındaki Kritik Fark

Stringler, karakter koleksiyonlarıdır; aritmetik işlemlere tabi tutulamazlar. Int ve double, matematiksel işlemler için tasarlanmıştır. Int sadece tam sayıları, double ise ondalıklı sayıları saklayarak farklı hassasiyet seviyeleri sunar.

Veri Tipleri Örneği: Alan Hesaplama

Dairenin alanını hesaplamak (`area()`), değerinin ondalıklı olması ve sonucun genellikle ondalıklı çıkması nedeniyle 'double' tipini kullanmayı zorunlu kılar. Eğer `int` kullanırsak, sonuç yuvarlanarak yanlış hesaplanabilir.

Operatörler: Aritmetik İşlemler

Dört temel aritmetik operatör: Toplama (+), Çıkarma (-), Çarpma (*) ve Bölme (/).

Bu operatörler, sayısal değişkenler (int veya double) üzerinde çalışır. İşlem önceliği (parantez, çarpma/bölme, toplama/çıkarma) matematiktekiyle aynıdır.



Operatörler: Metin Birleştirme (+)

Aynı '+' operatörü, string değişkenler arasında kullanıldığında metinleri uç uca ekler (birleştirir).

Sayısallar ile stringler birleştirildiğinde, sayısal değer otomatik olarak string'e dönüştürülür ve birleştirme işlemi yapılır.

Bölüm IV: Zorunlu Tip Dönüşümü (Type Conversion)

TextBox.Text özelliği daima string döndürür.

Ancak matematiksel işlemler (toplama, çıkarma) sadece sayısal tipler (int, double) üzerinde yapılabilir.

Bu nedenle, metin kutusundan okunan stringin sayısal işleme girmeden önce dönüştürülmesi zorunludur.



Dönüşüm Yöntemi 1: Convert Sınıfı

Convert sınıfı, bir veri tipini başka bir veri tipine güvenli bir şekilde dönüştürmek için geniş bir metot seti sunar.

Örn: `Convert.ToDouble()` veya `Convert.ToInt32()`.

Bu metotlar, dönüştürme başarısız olursa (örneğin metin girilirse) bir hata (exception) fırlatır, bu da try-catch bloklarını gerektirir.

Convert.ToDouble Kullanımı

```
double fiyat = Convert.ToDouble(txtFiyat.Text);
```

Bu komut, txtFiyat TextBox'ının içeriğini okur ve onu ondalıklı sayı tipine dönüştürerek 'fiyat' değişkenine atar.

Ondalık ayırıcı konusunda yerel ayarlara dikkat edilmelidir (genellikle nokta kullanılır).



Convert.ToInt32 Kullanımı

```
int adet = Convert.ToInt32(txtAdet.Text);
```

Bu komut, txtAdet TextBox'ının içeriğini okur ve onu tam sayı tipine dönüştürerek 'adet' değişkenine atar.

Eğer girilen metin ondalık içeriyorsa, bu dönüşüm hata verebilir.

Dönüşüm Yöntemi 2: Parse Metodu

Her temel sayısal veri tipi (int, double, vb.) kendi içinde bir 'Parse' metodu barındırır (Örn: `int.Parse()`, `double.Parse()`).

Parse metotları, string girdiyi doğrudan o tipin nesnesine dönüştürmeye çalışır.

Convert'e benzer şekilde, hatalı girdi durumunda exception fırlatır.



Dönüşümün Ters Yönü: Sayıdan Metne

Hesaplanan sayısal sonuçların (int, double) kullanıcıya gösterilebilmesi için Label.Text veya TextBox.Text özelliklerine atanmadan önce tekrar string'e dönüştürülmesi gerekir.

Bunun için tüm C# tiplerinin desteklediği 'ToString()' metodu kullanılır.

Formatlama: ToString(0.00)

Sayısal sonuçları, özellikle para birimi gibi alanlarda, standart bir formatta göstermek önemlidir.

ToString(0.00) ifadesi, sayıyı virgülden sonra zorunlu olarak iki basamakla formatlar.

Örn: 15.5'i '15.50 TL' olarak gösterir; bu, profesyonel sunum için gereklidir.

Bölüm V: Hata Yönetimi (Exception Handling)

Kullanıcılar her zaman beklenen tipte veri girmezler (örneğin, sayı beklerken metin girerler).

Convert veya Parse işlemleri hatalı girdi aldığı anda 'programı çökerten' bir istisna (exception) fırlatır.

Hata yönetiminin amacı, bu istisnaları yakalamak ve programın kullanıcı dostu bir şekilde çalışmaya devam etmesini sağlamaktır.



Try-Catch Bloğu: Tanım ve Yapı

try-catch bloğu, potansiyel olarak hata oluşturabilecek kodları 'try' bloğu içine alır.

Eğer try bloğunda bir hata oluşursa, program çökmez; onun yerine akış hemen 'catch' bloğuna geçer.

Catch bloğu, hatanın yönetilmesi veya kullanıcıya bilgi verilmesi gereken yeri içerir.

Try Bloğunun Amacı

Tüm veri okuma ve dönüştürme işlemleri, try bloğu içine yerleştirilmelidir.
Örn: TextBox'tan veri okuma, matematiksel formüller ve veriyi sayısal olarak kullanma adımları.
Bu sayede, kullanıcı hatası nedeniyle oluşabilecek istisnalar yakalanır.



Catch Bloğunun Görevi

Catch bloğu içinde genellikle kullanıcıya hatanın ne olduğu hakkında bilgi veren bir mesaj gösterilir.

MessageBox.Show(Lütfen geçerli sayılar giriniz.) yaygın kullanılan bir yöntemdir.

Bu, programın dostça bir şekilde kapanmasını veya devam etmesini sağlar.

Neden Try-Catch Kullanımı Zorunlu?

Sağlam programlar, beklenmeyen girdilere karşı dayanıklı olmalıdır. Veri girişi her zaman dışarıdan geldiği için güvenilmezdir; try-catch, bu dış tehditlere karşı bir koruma katmanı oluşturur. Özellikle ticari uygulamalarda çökme riskini en aza indirmek esastır.



Hata Türleri: FormatException

En yaygın hata, kullanıcı sayısal alanlara metin girdiğinde ortaya çıkan 'FormatException'dır.

Convert.ToDouble veya int.Parse metotları, stringin hedef formatla eşleşmediğini gördüğünde bu hatayı fırlatır.



Bölüm VI: Uygulama Örneği: Basit Sipariş Formu

Bu örnek proje (KDV Hesaplama), öğrenilen tüm kavramları (Kontroller, Değişkenler, Dönüşüm, Hata Yönetimi) bir araya getiren bütünleşik bir uygulamadır.

Senaryo: Kullanıcının girdiği ürün fiyatı ve adet bilgisine göre KDV dahil Genel Toplamı hesaplamak.



Sipariş Formu Tasarımı: Girdi Kontrolleri

Gerekli Kontroller:

'Ürün Fiyatı:' için Label ve txtFiyat (TextBox).

'Adet:' için Label ve txtAdet (TextBox).

Hesaplamayı başlatmak için btnHesapla (Button).

Sipariş Formu Tasarımı: Çıktı Kontrolleri

Sonuçların gösterilmesi için statik Label'lar ve sonuçları tutacak Label'lar gerekir:

lblAraToplam, lblKDV, lblGenelToplam Label'ları sonuçları gösterecektir. Bu Label'ların başlangıç 'Text' özelliği genellikle boş veya '0.00 TL' olarak ayarlanır.



Kodlama Adımı 1: Veri Okuma ve Dönüştürme

Kodlamaya başlarken ilk adım, kullanıcıdan alınan string verileri Convert kullanarak sayısal tiplere (double ve int) dönüştürmektir.

Bu adım try bloğu içinde yer almalıdır.

```
double fiyat = Convert.ToDouble(txtFiyat.Text);  
int adet = Convert.ToInt32(txtAdet.Text);
```

Kodlama Adımı 2: Hesaplamaların Yapılması

Ara Toplam: fiyat * adet.

KDV Tutarı: araToplam * 0.18 (KDV %18 oranı).

Genel Toplam: araToplam + kdvTutari.

Bu işlemler double tipleri üzerinde gerçekleştirilir.



Kodlama Adımı 3: Sonuçların Yazdırılması ve Formatlama

Hesaplanan double değerler, ToString(0.00) metodu kullanılarak string'e dönüştürülmeli ve formatlanmalıdır.

```
lblAraToplam.Text = araToplam.ToString( 0.00 ) + " TL ;
```

Aynı işlem KDV tutarı ve Genel Toplam için de tekrarlanır.

Kodlama Adımı 4: Hata Yönetimi

Eğer kullanıcı sayı yerine metin girerse, catch bloğu tetiklenir.
Catch bloğunun temel görevi, kullanıcıya kibar bir uyarı mesajı göstermektir:
`MessageBox.Show( Lütfen fiyat ve adet alanlarına geçerli sayılar giriniz. );`

Akademik Detay: Kltrel Dnm Farkları

Trkiye'de ondalık ayıracı olarak virgl (,) kullanılırken, uluslararası sistemlerde (ve Visual Studio/C# varsayılanında) nokta (.) kullanılır. 'Ondalıkly sayılar iin nokta yerine virgl kullanma' yaygın hatalardan biridir. Gerek uygulamalarda, kullanıcının yerel ayarlarını dikkate alan dnm metotları kullanılmalıdır (TryParse veya CultureInfo).



Veri Tipi Derinliđi: Float vs. Decimal

Kaynakta 'double' kullanılmasına rađmen, mali hesaplamalar (fiyat, KDV) iin genellikle 'decimal' tipi nerilir.

Decimal, daha az hız pahasına, ondalıklı sayılarda ikili kayan nokta (binary floating point) hatalarını nlemek iin daha yksek hassasiyet sunar.



Veri Girişi Kontrolü: TryParse

Convert/Parse, hata durumunda program akışını durdururken (exception fırlatırken), TryParse metodu daha zarif bir çözüm sunar. TryParse, dönüşümün başarılı olup olmadığını kontrol eden boolean bir değer döndürür ve hata fırlatmaz. Üniversite düzeyinde daha temiz hata yönetimi için bu metot önerilir.



Debugging Temelleri: Hata Tespiti

Öğrencilerin eksiklerini tespit etmeleri için hata ayıklama (debugging) sürecini öğrenmeleri önemlidir.

Breakpoint koyma, kodun adım adım çalıştırılması ve değişkenlerin anlık değerlerinin izlenmesi temel becerilerdir.

Bu, mantıksal hataları ve beklenmeyen dönüşüm sorunlarını bulmaya yardımcı olur.



Programlama Standartları: Kod Okunabilirliği

Üniversite projelerinde bile değişken isimlerinin anlamlı olması (txtFiyat, araToplam gibi) önemlidir.
Kod bloklarının (try-catch, if-else) doğru girintilenmesi ve düzenli yorum satırları eklenmesi, projenin sürdürülebilirliğini artırır.

Özet: Temel Kavramların Geri Çağırılması

IDE: Temel panellere ve kontrol eklemeye hakim misiniz?

Kontroller: Name, Text özellikleri ve Label/TextBox farkını biliyor musunuz?

Olaylar: Click olayının kod akışını nasıl başlattığını anladınız mı?

Tipler: String, int ve double arasındaki farkı ayırt edebiliyor musunuz?



Özet: Kritik Uygulama Becerileri

Dönüşüm: TextBox'tan okunan string'i Convert ile sayıya dönüştürebiliyor musunuz?

Formatlama: Sonuçları ToString(0.00) kullanarak formatlayabiliyor musunuz?

Hata Yönetimi: Try-catch bloğunu doğru yerde ve doğru amaçla kullanıyor musunuz?



Soru Çözümü 1: Metin Birleştirme

Senaryo: Kullanıcının girdiği 'Ad' ve 'Soyad' metinlerini birleştirip bir Label'da gösteren kod nasıl yazılır?

Çözüm: `lblSonuc.Text = txtAd.Text + " " + txtSoyad.Text;`

Önemli Nokta: Stringler arasında bile boşluk eklemek için araya `" "` string'i eklenmelidir.

Soru Çözümü 2: Daire Alanı Hesaplama

Senaryo: Kullanıcının girdiği yarıçap değerine göre dairenin alanını hesaplayan bir program nasıl yapılır?

Gereksinimler: double kullanımı, Convert.ToDouble, matematiksel formülün koda dökülmesi.

Alan = $\pi * r * r$. Math.PI sabitini kullanmak hassasiyeti artırır.

Soru Çözümü 3: Hata Yönetimi Senaryosu

Senaryo: Bir hesaplama butonuna basıldığında, kullanıcı fiyat alanına 'abc' girerse ne olur?

Cevap: FormatException hatası oluşur ve try-catch yoksa program çöker.

Çözüm: Programın çökmesini engellemek için tüm dönüşüm ve hesaplama kodlarının try bloğu içine alınması gerekir.



İpuçları ve Yaygın Hatalar (Eğitmen Rolü)

En yaygın hata: Sayısal işlem yapılması gereken yerde TextBox.Text'i Convert etmeyi unutmak.

İkinci yaygın hata: Ondalıklı sayılar için yerel ayar farkını göz ardı etmek (virgül/nokta).

Öğrenciler soru sormaktan çekinmemeli ve anlamadıkları konuları tekrar gözden geçirmelidir.



İleri Konulara Giriş: Nesne Yönelimli Programlama (OOP)

Bu hafta gördüğümüz kontroller (Label, Button) aslında C# dilinde birer sınıftır ve Nesne Yönelimli Programlamanın (OOP) temelini oluşturur.

OOP: Kapsülleme, Kalıtım, Polimorfizm ve Soyutlama gibi kavramlar üzerine kuruludur.

Kontrollerin 'özellikleri' (Properties) ve 'olayları' (Events), bu nesne tabanlı yapının somut örnekleridir.

Kapsülleme (Encapsulation) Örneği

Bir TextBox'ın 'Text' özelliği, dışarıdan erişilebilen ve değiştirilebilen bir veriyi temsil eder.

Ancak TextBox'ın dahili mantığı (örneğin metnin bellekte nasıl depolandığı) kullanıcıdan gizlenmiştir.

Bu, kapsüllemenin temel prensibidir: Veriyi korumak ve yalnızca kontrollü arayüzler (özellikler) aracılığıyla erişim sağlamak.



İpuçları: Etkili Çalışma Yöntemleri

Tekrar projeleri sıfırdan yazın, sadece kopyala-yapıştır yapmayın.
Her bir kod satırının ne yaptığını yüksek sesle açıklayın.
Örnek senaryoları kendiniz değiştirin (Örn: KDV'yi %18'den %20'ye çevirin) ve kodunuzun hala doğru çalışıp çalışmadığını test edin.

Görsel Programlama Vize Tekrarı: Sonuç

Bu hafta, ilk altı haftanın kritik konularını pekiştirerek vizeye hazır hale gelmeniz hedeflenmiştir.

Başarılı bir Görsel Programlama öğrencisi, sadece kod yazmakla kalmaz, aynı zamanda kullanıcı deneyimi ve hata yönetimi konularında da duyarlı olmalıdır.

Tekrar pratik yapmaya devam edin!



Görsel Programlama Vize Tekrarı ve Pekiştirme

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

